

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



BÁO CÁO CUỐI KÌ
INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Đề tài:
Xây dựng mô hình Large Language Learning (LLM)

Sinh viên thực hiện: Lê Quang Huy - 23001885
Đỗ Tùng Dương – 23001858
Hoàng Trọng Đại – 23001860
Mã học phần: MAT1206E
Giảng viên hướng dẫn: TS. Nguyễn Hải Vinh

HÀ NỘI, 12/2025

Mục lục

Lời nói đầu	3
1 Giới thiệu tổng quan	4
1.1 Giới thiệu về đề tài	4
1.2 Lý do chọn đề tài	4
1.3 Các nội dung dự kiến thực hiện	4
2 Cơ sở lý thuyết	5
2.1 Tensor	5
2.2 Gradient Descent	8
2.3 Pytorch	9
2.4 Mô hình ngôn ngữ (Language Model)	10
2.5 Embedding (Token Embedding & Positional Embedding)	12
2.6 Self-Attention và Multi-Head Attention	15
2.7 Transformer	19
3 Mô tả bài toán	22
3.1 Bài toán sinh văn bản tự động	22
3.2 Dữ liệu huấn luyện	22
3.3 Character-level prediction	23
4 Thiết kế hệ thống và mô hình	24
4.1 Pipeline tổng quát	24
4.2 Tiền xử lý dữ liệu	24
4.3 Kiến trúc mô hình GPT Mini	25
4.4 Siêu tham số	26
5 Huấn luyện, thực nghiệm và đánh giá	28
5.1 Quy trình huấn luyện	28
5.2 Lập trình training loop và batch generation	29
5.3 Đánh giá loss định kỳ	30
5.4 Kết quả huấn luyện và sinh văn bản	31

5.5	Kết luận và hướng phát triển	32
	Tài liệu tham khảo	35

Lời nói đầu

Trong những năm gần đây, sự phát triển mạnh mẽ của Trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn (Large Language Models – LLM), đã tạo ra bước tiến vượt bậc trong khả năng xử lý và hiểu ngôn ngữ tự nhiên của máy tính. Các mô hình như GPT, LLaMA, Mistral hay Qwen không chỉ đóng vai trò nền tảng cho nhiều ứng dụng thông minh mà còn trở thành chủ đề nghiên cứu quan trọng trong lĩnh vực học sâu và xử lý ngôn ngữ tự nhiên.

Đối với sinh viên Công nghệ thông tin, việc tiếp cận và hiểu rõ cách thức hoạt động của các mô hình này không chỉ mở rộng kiến thức chuyên ngành mà còn giúp xây dựng nền tảng vững chắc cho các nghiên cứu và ứng dụng nâng cao trong tương lai. Xuất phát từ ý nghĩa đó, nhóm thực hiện đề tài "Xây dựng và huấn luyện mô hình ngôn ngữ LLM dạng GPT từ đầu bằng PyTorch" như một phần của báo cáo cuối kỳ học phần Nhập môn Trí tuệ nhân tạo.

Báo cáo này tập trung trình bày quá trình xây dựng một mô hình GPT đơn giản từ mức độ thấp nhất: xử lý dữ liệu đầu vào, thiết kế các thành phần cốt lõi của Transformer (Self-Attention, Multi-Head Attention, FeedForward, LayerNorm, Embedding), huấn luyện mô hình trên tập dữ liệu văn bản, và đánh giá khả năng sinh văn bản của mô hình. Qua đó, nhóm mong muốn làm rõ các nguyên lý nền tảng tạo nên một mô hình LLM hiện đại, đồng thời giúp người đọc có cái nhìn trực quan và hệ thống về cách hoạt động của kiến trúc này.

1. Giới thiệu tổng quan

1.1. Giới thiệu về đề tài

Đề tài tập trung xây dựng một mô hình ngôn ngữ dạng GPT thu nhỏ bằng PyTorch nhằm tìm hiểu cách hoạt động của các mô hình ngôn ngữ lớn (LLM). Mô hình gồm các thành phần cơ bản của Transformer như Embedding, Position Embedding, Multi-Head Self-Attention, Feed-Forward Network và cơ chế sinh văn bản theo hướng autoregressive. Dữ liệu huấn luyện được lấy từ văn bản "The Wizard of Oz", xử lý bằng memory map để tối ưu tốc độ truy cập. Đề tài giúp sinh viên nắm được nguyên lý của LLM thông qua việc tự triển khai và huấn luyện mô hình từ đầu.

1.2. Lý do chọn đề tài

LLM hiện là xu hướng trọng tâm của trí tuệ nhân tạo và có vai trò quan trọng trong học tập cũng như nghiên cứu. Việc tự xây dựng một mô hình GPT mini giúp sinh viên hiểu rõ cơ chế Attention và kiến trúc Transformer thay vì chỉ sử dụng API có sẵn. Mô hình thu nhỏ phù hợp với tài nguyên của máy cá nhân nhưng vẫn thể hiện đầy đủ cấu trúc cơ bản của một LLM. Đồng thời, đề tài còn giúp sinh viên rèn luyện kỹ năng lập trình PyTorch, xử lý dữ liệu và huấn luyện mô hình, phù hợp với yêu cầu của môn học.

1.3. Các nội dung dự kiến thực hiện

Trong quá trình triển khai đề tài, nhóm dự kiến thực hiện các bước chính sau: tìm hiểu các kiến thức cơ bản về Transformer và mô hình ngôn ngữ; chuẩn bị dữ liệu, xây dựng vocab và tokenizer; cài đặt mô hình GPT mini bao gồm các thành phần Attention, Feed-Forward, Block và Embedding; xây dựng hàm huấn luyện, đánh giá và sinh văn bản; tiến hành huấn luyện mô hình trên dữ liệu "The Wizard of Oz"; đánh giá kết quả, lưu mô hình; và cuối cùng là thử nghiệm sinh văn bản từ prompt để kiểm tra chất lượng mô hình.

2. Cơ sở lý thuyết

2.1. Tensor

2.1.1. Định nghĩa Tensor

Tensor là một cấu trúc dữ liệu toán học dùng để biểu diễn thông tin trong không gian nhiều chiều và đóng vai trò nền tảng trong hầu hết các mô hình học sâu. Tensor có thể được xem như sự mở rộng của các dạng dữ liệu quen thuộc trong đại số tuyến tính.

Trước hết, một giá trị vô hướng (scalar) được xem là tensor bậc 0, một vector là tensor bậc 1, và một ma trận là tensor bậc 2. Các tensor bậc cao hơn (từ bậc 3 trở lên) được sử dụng để biểu diễn các loại dữ liệu có cấu trúc phức tạp hơn như chuỗi thời gian, dữ liệu âm thanh, ảnh nhiều kênh hoặc dữ liệu video.

Mỗi tensor được mô tả bởi ba đặc trưng chính: bậc (rank) – thể hiện số chiều của tensor; kích thước (shape) – mô tả số phần tử trong từng chiều; và kiểu dữ liệu (dtype) như float32 hoặc int64. Những đặc trưng này giúp mô tả đầy đủ cấu trúc và loại dữ liệu mà tensor chứa.

Trong học sâu, tất cả dữ liệu đầu vào, trọng số của mô hình cũng như các giá trị trung gian trong quá trình tính toán đều được biểu diễn dưới dạng tensor. Điều này giúp tận dụng khả năng xử lý song song của GPU, từ đó tăng tốc đáng kể các phép toán trên dữ liệu nhiều chiều và làm cho các mô hình học sâu hoạt động hiệu quả hơn.

Một số ví dụ:

- **Scalar:**

$$x = 5$$

- **Vector:**

$$v = [1, 2, 3] \quad (\text{shape} = (3))$$

- **Matrix:**

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad (\text{shape} = (2, 2))$$

- **Tensor 3 chiều (ảnh màu RGB):**

$$\text{Image Tensor: } (\text{Height}, \text{Width}, \text{Channels}) = (224, 224, 3)$$

2.1.2. Các phép toán với Tensor

Các tensor hỗ trợ nhiều loại phép toán khác nhau, cho phép xử lý dữ liệu đa chiều một cách hiệu quả và nhất quán.

1. **Phép toán phần tử:** Các phép toán này thực hiện trực tiếp lên từng phần tử tương ứng của hai tensor có cùng kích thước.

- Phép cộng:

$$C = A + B, \quad C_{i,j} = A_{i,j} + B_{i,j}$$

- Phép trừ:

$$C = A - B, \quad C_{i,j} = A_{i,j} - B_{i,j}$$

- Phép nhân từng phần tử:

$$C = A \odot B, \quad C_{i,j} = A_{i,j} \cdot B_{i,j}$$

2. **Phép nhân ma trận:**

$$C = AB, \quad C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

3. **Phép nhân vô hướng:**

$$a \cdot b = \sum_{i=1}^n a_i b_i$$

4. **Chuẩn của Tensor:**

- Chuẩn L2:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Chuẩn L1:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|$$

- Chuẩn Frobenius (cho ma trận hoặc tensor bậc 2):

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n A_{ij}^2}$$

5. Các phép toán giảm chiều:

- Tổng tất cả các phần tử của tensor $\mathbf{A}$:

$$\text{sum}(\mathbf{A}) = \sum_{i,j,\dots} A_{ij\dots}$$

- Trung bình các phần tử của tensor:

$$\text{mean}(\mathbf{A}) = \frac{1}{N} \sum_{i,j,\dots} A_{ij\dots}$$

- Giá trị lớn nhất/nhỏ nhất:

$$\max(\mathbf{A}) = \max_{i,j,\dots} A_{ij\dots}$$

$$\min(\mathbf{A}) = \min_{i,j,\dots} A_{ij\dots}$$

2.1.3. Ứng dụng của Tensor

1. **Xử lý hình ảnh:** Hình ảnh được biểu diễn dưới dạng tensor bậc 3 hoặc bậc 4 (ví dụ: chiều cao · chiều rộng · kênh màu, hoặc thêm batch size), giúp các mô hình CNN xử lý nhiều ảnh cùng lúc, trích xuất đặc trưng và nhận diện hình ảnh.
2. **Xử lý văn bản và ngôn ngữ tự nhiên:** Trong các mô hình ngôn ngữ lớn (LLM), từ ngữ, câu, hoặc embedding từ được biểu diễn dưới dạng tensor. Các phép toán tensor được dùng để tính attention, softmax, hay chuyển đổi embedding, từ đó mô hình học cách dự đoán từ tiếp theo hoặc sinh văn bản.
3. **Xử lý âm thanh và chuỗi thời gian:** Dữ liệu âm thanh, tín hiệu cảm biến, hoặc chuỗi thời gian được lưu trữ dưới dạng tensor nhiều chiều, giúp các mô hình RNN, LSTM, Transformer có thể học các mối quan hệ theo thời gian.

4. **Học máy và học sâu tổng quát:** Các mạng nơ-ron, từ mạng tuyến tính đơn giản đến mạng sâu phức tạp, đều sử dụng tensor để biểu diễn đầu vào, trọng số, gradient, và output. Nhờ tensor, các phép toán gradient descent và backpropagation có thể thực hiện hiệu quả, đặc biệt khi tận dụng GPU.

2.2. Gradient Descent

2.2.1. Định nghĩa Gradient Descent

Gradient Descent là một thuật toán tối ưu hóa cơ bản được sử dụng rộng rãi trong học máy và học sâu nhằm tìm giá trị tối ưu của các tham số mô hình bằng cách giảm thiểu hàm mất mát (loss function). Thuật toán dựa trên việc tính gradient – đạo hàm riêng của hàm mất mát theo các tham số – để xác định hướng dốc giảm nhanh nhất và cập nhật các tham số theo hướng ngược lại.

Công thức cập nhật tham số trong Gradient Descent được biểu diễn như sau:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

Trong đó:

- $\mathbf{w}^{(t)}$ là vector tham số tại bước t ,
- η là learning rate (tốc độ học),
- $\nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$ là gradient của hàm mất mát theo tham số $\mathbf{w}$ tại bước t .

2.2.2. Nguyên lý hoạt động của Gradient Descent

1. **Khởi tạo tham số:** Thuật toán bắt đầu với một tập hợp các tham số $\mathbf{w}^{(0)}$, thường được khởi tạo ngẫu nhiên.
2. **Tính gradient:** Gradient của hàm mất mát $L(\mathbf{w})$ theo các tham số $\mathbf{w}$ được tính tại bước hiện tại. Gradient này cho biết hướng tăng nhanh nhất của hàm mất mát.

$$\nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

3. **Cập nhật tham số:** Tham số được điều chỉnh theo hướng ngược lại gradient, với bước nhảy được điều khiển bởi learning rate η :

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}^{(t)})$$

4. **Lặp lại quá trình:** Các bước tính gradient và cập nhật tham số được lặp lại cho đến khi hàm mất mát hội tụ hoặc đạt số vòng lặp tối đa.

2.3. Pytorch

2.3.1. Giới thiệu về Pytorch

PyTorch là một framework học máy dựa trên thư viện Torch, được sử dụng trong lĩnh vực thị giác máy tính và xử lý ngôn ngữ tự nhiên, do Meta AI phát triển và ngày nay là một phần của Linux Foundation. Đây là phần mềm tự do nguồn mở phát hành dựa trên giấy phép BSD đã qua sửa đổi. Được phát triển chính yếu ở dạng giao diện ngôn ngữ Python

2.3.2. Kiến trúc tổng quan trong Pytorch

1. **Tensor trong Pytorch:** Tensor là cấu trúc dữ liệu cơ bản trong PyTorch, đóng vai trò tương tự như mảng đa chiều của NumPy nhưng mạnh hơn nhờ khả năng chạy trên CPU lẫn GPU.
2. **Autograd:** Autograd là cơ chế tự động tính đạo hàm trong PyTorch. Khi thực hiện các phép toán trên Tensor có `requires_grad = True`, PyTorch sẽ xây dựng một đồ thị tính toán (computational graph). Từ đó, khi gọi `backward()`, hệ thống sẽ tự động suy ngược để tính gradient cho từng biến.

Cơ chế này đặc biệt quan trọng trong huấn luyện mô hình deep learning vì gradient là yếu tố quyết định cho thuật toán tối ưu (như Gradient Descent).

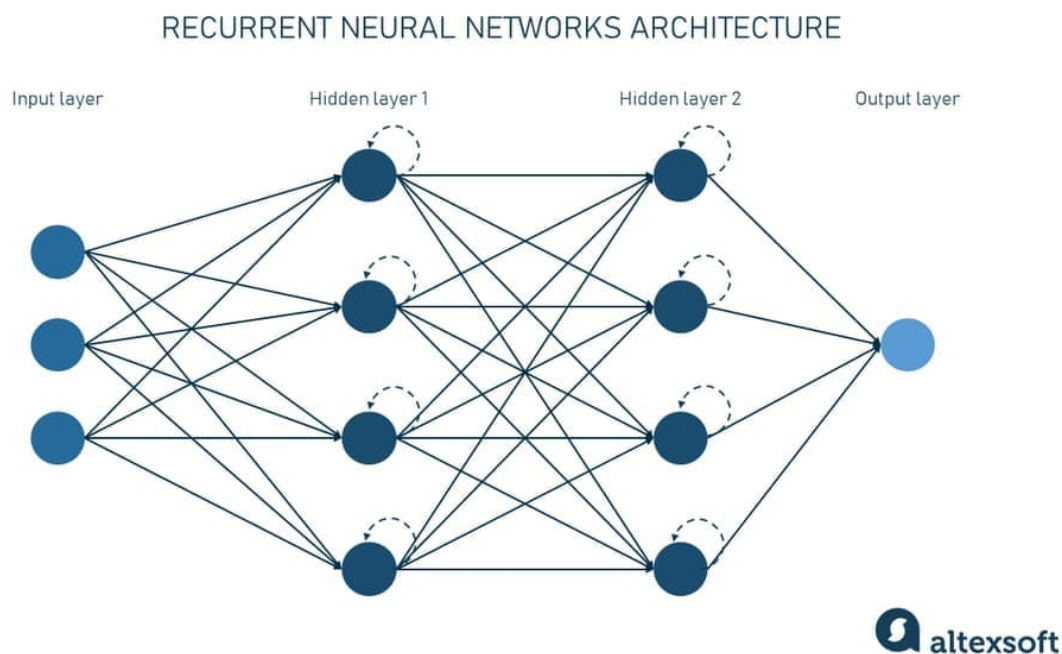
3. **Deep Learning Library:** PyTorch không chỉ là một thư viện tính toán tensor, mà còn là một nền tảng mạnh mẽ phục vụ phát triển các mô hình học sâu (Deep Learning). Deep Learning Library của PyTorch được thiết kế theo hướng linh hoạt, trực quan, dễ kiểm thử

và đặc biệt phù hợp cho các mô hình hiện đại như CNN, RNN, Transformer và các mô hình ngôn ngữ lớn (LLM).

Hệ sinh thái Deep Learning trong PyTorch bao gồm nhiều module quan trọng, hỗ trợ toàn bộ pipeline phát triển mô hình từ xử lý dữ liệu, xây dựng kiến trúc mạng, lan truyền ngược, tối ưu hóa cho đến triển khai mô hình.

2.4. Mô hình ngôn ngữ (Language Model)

Mô hình ngôn ngữ là một thành phần quan trọng trong xử lý ngôn ngữ tự nhiên, có nhiệm vụ ước lượng xác suất xuất hiện của một chuỗi ký tự hoặc từ. Nói cách khác, mô hình học cách dự đoán token tiếp theo dựa trên các token trước đó trong một chuỗi văn bản.



Hình 2.1: Language Model

2.4.1. Mục tiêu và nguyên lý

Trong dự án GPT mini, mô hình ngôn ngữ được huấn luyện theo character-level, tức là dự đoán từng ký tự tiếp theo dựa trên bối cảnh các ký tự trước đó. Mục tiêu của mô hình là học được

phân phối xác suất của các ký tự, từ đó có thể sinh ra văn bản tự động.

Công thức cơ bản của mô hình ngôn ngữ:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

Tại mỗi thời điểm t , mô hình dự đoán xác suất xuất hiện của ký tự tiếp theo dựa trên các ký tự đã xuất hiện. Đây là cơ sở để mô hình có thể sinh văn bản mới dựa trên đoạn đầu vào (prompt).

2.4.2. Loại mô hình ngôn ngữ

Có nhiều loại mô hình ngôn ngữ, từ truyền thống đến hiện đại. Các mô hình N-gram dự đoán token tiếp theo dựa trên $n - 1$ token trước đó, là mô hình xác suất đơn giản nhưng hạn chế trong việc xử lý chuỗi dài do bối cảnh bị giới hạn. Các mô hình tuần tự như RNN, LSTM, GRU có khả năng ghi nhớ thông tin trong chuỗi dài hơn so với N-gram, tuy nhiên quá trình huấn luyện dễ gặp vấn đề vanishing hoặc exploding gradient. Trong khi đó, các mô hình dựa trên Transformer, ví dụ GPT và BERT, sử dụng cơ chế self-attention, cho phép mô hình học mối quan hệ giữa tất cả các token trong chuỗi cùng một lúc. GPT là một mô hình ngôn ngữ autoregressive, dự đoán token tiếp theo dựa trên bối cảnh các token trước đó.

2.4.3. Mô hình ngôn ngữ character-level và word-level

Mô hình ngôn ngữ có thể được xây dựng ở hai cấp độ là character-level và word-level. Ở character-level, mỗi token là một ký tự, mô hình dự đoán ký tự tiếp theo, phù hợp với các tập dữ liệu nhỏ và giúp tránh vấn đề từ hiếm (rare words). Ở word-level, mỗi token là một từ, mô hình học ngữ cảnh ở mức từ, thường cho kết quả sinh văn bản nhanh hơn và tự nhiên hơn, nhưng yêu cầu bộ từ vựng lớn và khả năng xử lý dữ liệu phức tạp hơn.

2.4.4. Vai trò trong dự án GPT mini

Trong dự án GPT mini, mô hình được huấn luyện theo character-level, tức là dự đoán từng ký tự tiếp theo trong văn bản. Mục tiêu của mô hình là học phân phối xác suất của từng ký tự dựa trên bối cảnh trước đó. Khi huấn luyện xong, mô hình có khả năng sinh văn bản tự động dựa

trên một đoạn văn bản đầu vào (prompt), đồng thời giữ được cấu trúc ngôn ngữ và các mối quan hệ giữa các ký tự trong câu.

2.5. Embedding (Token Embedding & Positional Embedding)

2.5.1 Token Embedding

Token Embedding là quá trình ánh xạ mỗi token rời rạc (trong trường hợp GPT mini là từng ký tự) sang một vector liên tục có kích thước cố định, gọi là embedding dimension. Mục đích của token embedding là biến dữ liệu văn bản từ dạng ký tự hoặc từ mà máy tính không thể xử lý trực tiếp thành các vector số học, giúp mô hình học được mối quan hệ ngữ nghĩa giữa các token.

Trong GPT mini, mỗi ký tự được biểu diễn bằng một vector có kích thước 128. Bảng embedding này được học tự động trong quá trình huấn luyện, nghĩa là mô hình sẽ điều chỉnh giá trị vector sao cho biểu diễn token phản ánh được ngữ cảnh của ký tự trong văn bản. Token embedding là bước đầu tiên trong pipeline của mô hình, cung cấp đầu vào có dạng liên tục cho các layer Transformer tiếp theo, cho phép mô hình thực hiện cơ chế self-attention và dự đoán token tiếp theo.

Token embedding giúp mô hình nhận biết và xử lý các token một cách số học, từ đó học được các mẫu ngôn ngữ, sự lặp lại và các mối quan hệ giữa các ký tự trong chuỗi văn bản.

2.5.2. Positional Embedding

Khái niệm: Positional Embedding là cơ chế để cung cấp thông tin về vị trí của từng token trong chuỗi đầu vào cho mô hình Transformer. Vì Transformer xử lý tất cả token song song và không có cơ chế tuần tự như RNN, nên cần một biểu diễn riêng cho vị trí của token, giúp mô hình biết thứ tự trong chuỗi.

Cách hoạt động:

- Mỗi vị trí trong chuỗi từ 0 đến `block_size - 1` được ánh xạ thành một vector liên tục có cùng kích thước với embedding token.
- Vector này là tham số học được trong quá trình huấn luyện, nghĩa là mô hình tự điều chỉnh các giá trị embedding vị trí để tối ưu cho nhiệm vụ dự đoán token tiếp theo.

- Khi tính đầu vào cho Transformer, embedding của token được cộng với embedding của vị trí tương ứng, tạo thành biểu diễn cuối cùng của token có thông tin cả về nội dung và vị trí.

Vai trò:

- Giúp mô hình nhận biết thứ tự các token trong chuỗi.
- Cho phép các cơ chế attention tính toán mối quan hệ giữa các token với nhận thức đầy đủ về vị trí tương đối, đảm bảo cấu trúc tuần tự trong chuỗi được duy trì.
- Là yếu tố quan trọng để mô hình học các mối quan hệ ngữ cảnh dài hạn trong văn bản.

Token Embedding vs. Positional Embedding

Feature	Token Embedding	Positional Encoding
Function	Captures semantic meaning	Captures word position
Learning	Learned from data	Predefined or learned
Focus	Word meaning	Word order and context
Benefits	Understands word relationships	Understands sentence structure
Limitations	Ignores word order	Can be computationally expensive

Hình 2.2: Token Embedding & Positional Embedding

2.5.3. Kết hợp Token và Positional Embedding

Mục tiêu: Tạo ra biểu diễn đầu vào đầy đủ cho Transformer, vừa chứa thông tin về nội dung của token, vừa chứa thông tin về vị trí của token trong chuỗi.

Cách thực hiện:

- **Token Embedding:** Mỗi token trong chuỗi được ánh xạ sang một vector liên tục, biểu diễn nội dung của token.
- **Positional Embedding:** Mỗi vị trí trong chuỗi được ánh xạ sang một vector liên tục, biểu diễn thông tin thứ tự của token.
- **Cộng hai loại embedding:**
 - Vector token và vector vị trí có cùng kích thước (embedding dimension).
 - Biểu diễn đầu vào cuối cùng cho mỗi token là tổng của token embedding và positional embedding.
 - Kết quả này được đưa vào các lớp Transformer tiếp theo để học mối quan hệ giữa các token, dựa trên cả nội dung và vị trí.

Lợi ích:

- Giữ nguyên thông tin ngữ nghĩa của token và thông tin thứ tự trong chuỗi.
- Cho phép cơ chế self-attention tính toán mối quan hệ giữa các token một cách chính xác theo vị trí.
- Là tiền đề quan trọng để mô hình GPT mini học phân phối xác suất của từng token tiếp theo dựa trên bối cảnh trước đó.

2.5.4. Ý nghĩa và vai trò trong mô hình GPT mini

Ý nghĩa:

- Embedding là bước biến đổi dữ liệu rời rạc (ký tự hoặc từ) thành dạng vector liên tục, cho phép mô hình học các đặc trưng ngữ nghĩa và cấu trúc của ngôn ngữ.

- Token embedding lưu giữ thông tin về nội dung của từng token.
- Positional embedding lưu giữ thông tin thứ tự, giúp mô hình nhận biết vị trí của token trong chuỗi.
- Việc kết hợp cả hai loại embedding tạo ra biểu diễn đầu vào giàu thông tin, đồng thời giữ được cả nội dung và thứ tự của văn bản.

Vai trò trong GPT mini:

- Cung cấp đầu vào chất lượng cho các block Transformer, giúp mô hình nhận biết mối quan hệ giữa các ký tự trong chuỗi.
- Cho phép cơ chế self-attention tính toán trọng số chính xác dựa trên bối cảnh, từ đó dự đoán token tiếp theo một cách hiệu quả.
- Là nền tảng để GPT mini học phân phối xác suất của từng ký tự và sinh văn bản tự động dựa trên prompt.

2.6. Self-Attention và Multi-Head Attention

2.6.1. Khái niệm Self-Attention

Khái niệm:

Self-Attention là cơ chế cho phép mô hình học được mối quan hệ giữa các token trong cùng một chuỗi đầu vào. Thay vì chỉ xử lý tuần tự từng token như RNN, self-attention cho phép mỗi token “chú ý” đến tất cả các token khác trong chuỗi, từ đó mô hình có thể nắm bắt các phụ thuộc dài hạn và ngữ cảnh phức tạp trong văn bản.

Cơ chế hoạt động:

Trong self-attention, mỗi token được biểu diễn bởi ba vector đặc trưng:

- **Query (Q):** đại diện cho token đang xét, dùng để truy vấn thông tin từ các token khác.
- **Key (K):** đại diện cho các token khác trong chuỗi, dùng để xác định mức độ liên quan với Query.

- **Value (V):** chứa thông tin thực tế của token, sẽ được trộn weighted để tạo đầu ra attention.

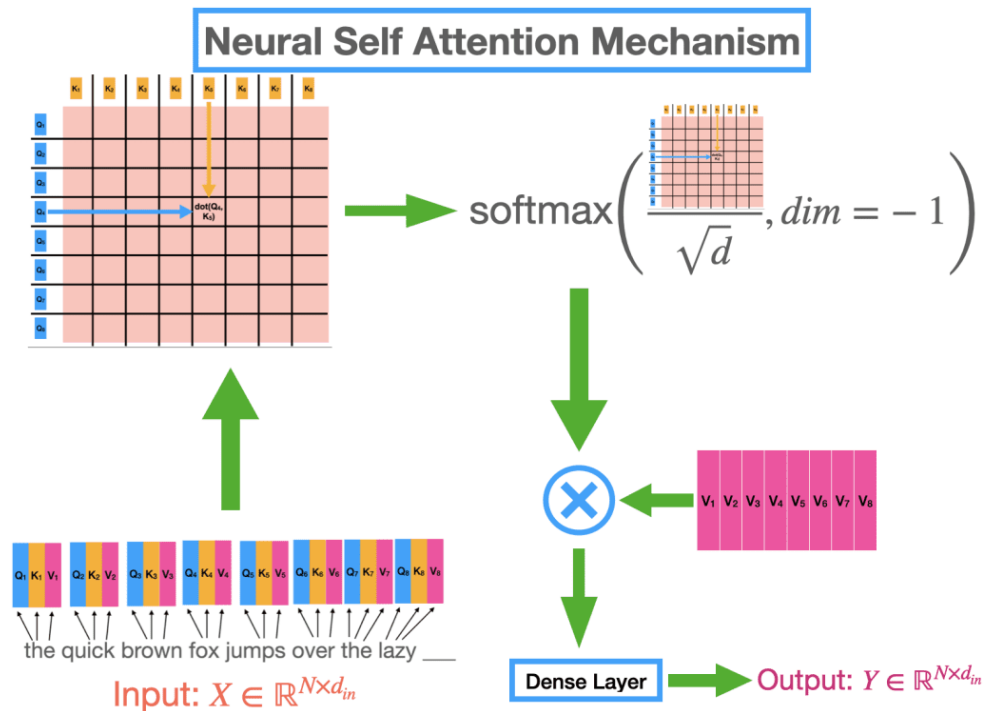
Attention được tính bằng cách đo mức độ tương đồng giữa Query và Key, sau đó chuẩn hóa bằng softmax để nhận được trọng số attention. Các trọng số này sau đó được nhân với Value để tổng hợp thông tin từ các token liên quan, tạo ra biểu diễn mới cho mỗi token, vừa chứa thông tin bản thân vừa phản ánh ngữ cảnh xung quanh.

Vai trò:

Self-attention giúp mô hình nắm bắt:

- Mọi quan hệ dài hạn giữa các token trong chuỗi, vượt quá giới hạn bối cảnh của n-gram.
- Ngữ cảnh và phụ thuộc phức tạp, như đồng âm, đại từ tham chiếu, hoặc cấu trúc câu dài.

Trong GPT mini, self-attention là nền tảng để mô hình dự đoán ký tự tiếp theo dựa trên toàn bộ bối cảnh trước đó trong một block văn bản.



Hình 2.3: Self-Attention

2.6.2. Cơ chế tính toán Self-Attention

1. **Sinh Q, K, V từ embedding đầu vào:** Mỗi token trong chuỗi đầu vào được biểu diễn dưới dạng embedding. Từ embedding này, mô hình tạo ra ba vector riêng biệt: Query (Q), Key (K) và Value (V) thông qua các phép biến đổi tuyến tính. Q dùng để truy vấn thông tin, K dùng để đo mức độ liên quan, và V chứa thông tin thực tế của token.
2. **Tính trọng số attention dựa trên dot-product giữa Q và K:** Mức độ liên quan giữa các token được xác định bằng tích vô hướng (dot-product) giữa Query của token hiện tại và Key của tất cả các token khác. Kết quả là một ma trận trọng số attention thô, thể hiện mức độ mà mỗi token nên "chú ý" tới các token khác.
3. **Áp dụng softmax để chuẩn hóa trọng số:** Ma trận trọng số thô được chuẩn hóa bằng hàm softmax theo từng hàng, đảm bảo rằng các trọng số attention nằm trong khoảng $[0, 1]$ và tổng bằng 1. Điều này giúp mô hình đánh giá tầm quan trọng tương đối của từng token trong bối cảnh.
4. **Tính toán weighted sum với V để tạo đầu ra attention:** Các trọng số attention sau khi chuẩn hóa được nhân với Value của các token tương ứng. Tổng trọng số này tạo ra đầu ra attention cho từng token, chứa thông tin bản thân token đồng thời tích hợp ngữ cảnh từ các token liên quan trong chuỗi.
5. **Khả năng nhìn toàn bộ chuỗi cùng lúc:** Nhờ cơ chế dot-product giữa Q và K với tất cả token, self-attention cho phép mô hình xem xét toàn bộ chuỗi đầu vào đồng thời. Mỗi token có thể tham chiếu đến bất kỳ token nào khác, giúp nắm bắt các phụ thuộc dài hạn, cấu trúc câu phức tạp, và các quan hệ ngữ nghĩa trong văn bản, vượt trội hơn so với các mô hình tuần tự truyền thống.

2.6.3. Multi-Head Attention

Khái niệm multi-head:

Multi-Head Attention là cơ chế mở rộng của self-attention, trong đó nhiều "head" attention hoạt động song song. Mỗi head thực hiện self-attention riêng biệt trên cùng một embedding đầu vào, cho phép mô hình học nhiều khía cạnh khác nhau của mối quan hệ giữa các token.

Lợi ích:

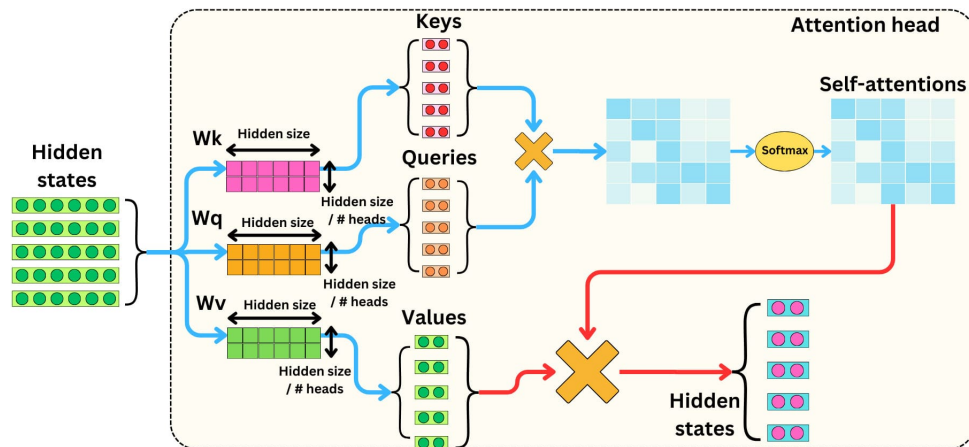
Sử dụng nhiều head giúp mô hình nắm bắt đa dạng các loại quan hệ giữa token trong cùng một chuỗi. Một head có thể tập trung vào phụ thuộc gần kề, head khác có thể nhận biết phụ thuộc dài hạn hoặc các cấu trúc cú pháp, ngữ nghĩa phức tạp. Nhờ đó, mô hình học được biểu diễn ngữ cảnh phong phú hơn.

Cách ghép và biến đổi kết quả:

Các đầu ra từ từng head được ghép lại theo chiều embedding, sau đó đi qua một lớp biến đổi tuyến tính (linear projection) để thu về kích thước embedding gốc. Kết quả cuối cùng vừa tổng hợp thông tin từ các head, vừa giữ đúng kích thước và định dạng cho các block Transformer tiếp theo.

Vai trò trong Transformer và GPT mini:

Multi-Head Attention là thành phần chủ chốt trong mỗi block Transformer. Trong GPT mini, cơ chế này cho phép mô hình dự đoán ký tự tiếp theo dựa trên toàn bộ bối cảnh trước đó, nắm bắt các quan hệ ngữ cảnh phong phú và cải thiện khả năng sinh văn bản tự nhiên.



Hình 2.4: Multi-Head Attention

2.6.4. Ý nghĩa và vai trò trong GPT mini

Tầm quan trọng cho dự đoán token tiếp theo:

Self-attention và Multi-Head Attention cho phép mô hình nhận biết và nắm bắt các mối quan hệ giữa tất cả các token trong chuỗi đầu vào. Nhờ đó, GPT mini có thể dự đoán token tiếp theo dựa trên toàn bộ bối cảnh trước đó, thay vì chỉ dựa vào một số token gần kề.

Khả năng học ngữ cảnh dài và quan hệ phức tạp:

Cơ chế attention giúp mô hình học được cả phụ thuộc gần và phụ thuộc dài hạn giữa các ký tự, cũng như các quan hệ cú pháp, ngữ nghĩa phức tạp. Multi-head attention tăng khả năng học đa dạng các loại quan hệ này đồng thời.

Nền tảng cho việc học phân phối xác suất và sinh văn bản:

Nhờ self-attention và multi-head attention, GPT mini có thể xây dựng phân phối xác suất chính xác cho từng token, từ đó sinh văn bản tự động một cách tự nhiên và giữ được cấu trúc ngôn ngữ cũng như mối quan hệ giữa các ký tự trong câu.

2.7. Transformer

2.7.1. Giới thiệu về Transformer

Transformer là một kiến trúc mạng nơ-ron sâu được giới thiệu bởi Vaswani và cộng sự năm 2017 trong công trình nổi tiếng “Attention Is All You Need”. Đây được xem là bước ngoặt quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), đặt nền móng cho các mô hình ngôn ngữ lớn (LLM) hiện đại như GPT, BERT, LLaMA, Gemini. . .

Khác với các mô hình truyền thống như RNN hoặc LSTM, vốn xử lý dữ liệu theo trình tự tuần tự (từ trái sang phải), Transformer hoạt động hoàn toàn dựa trên cơ chế attention và xử lý song song. Điều này cho phép mô hình học được mối quan hệ giữa các từ trong câu mà không bị giới hạn bởi khoảng cách, đồng thời rút ngắn đáng kể thời gian huấn luyện.

Transformer gồm hai thành phần chính là Encoder và Decoder.

- **Encoder:** dùng để hiểu và biểu diễn ngữ nghĩa của câu đầu vào.
- **Decoder:** dùng để sinh dữ liệu hoặc dự đoán token tiếp theo.

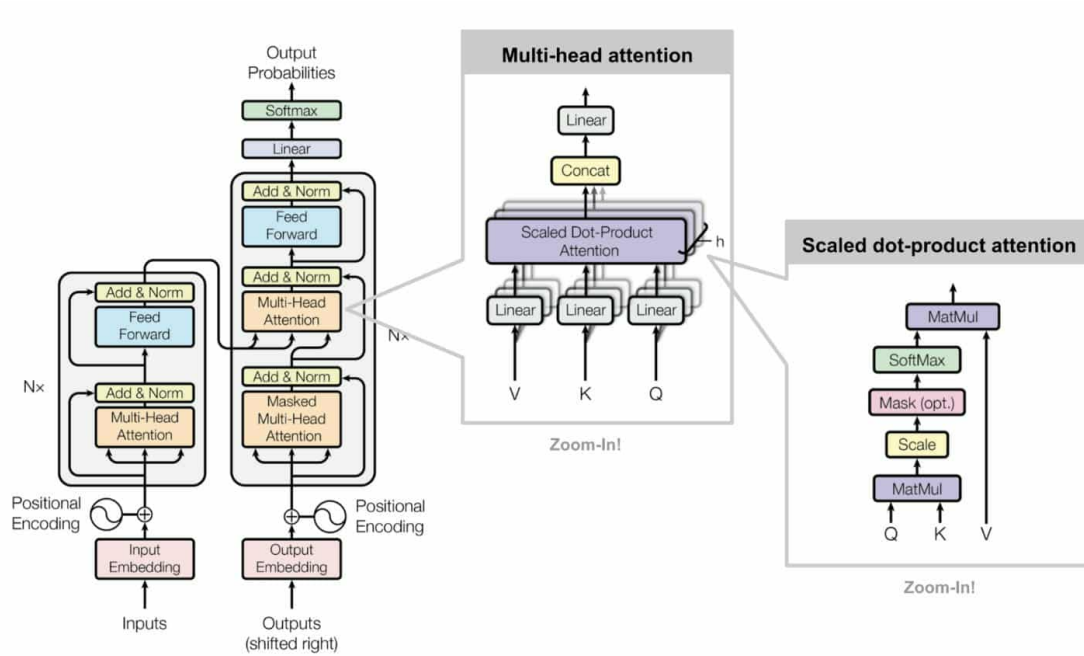
Điểm cốt lõi làm nên sức mạnh của Transformer là cơ chế Self-Attention – cho phép mô hình xác định từ nào trong câu quan trọng hơn và cần tập trung vào khi xử lý. Nhờ cơ chế này, Transformer có thể hiểu các phụ thuộc dài (long-range dependencies), ví dụ như mối liên hệ giữa chủ ngữ ở đầu câu và động từ ở cuối câu.

Với khả năng xử lý song song, hiểu ngữ cảnh tốt và dễ mở rộng quy mô, Transformer nhanh chóng trở thành kiến trúc chủ đạo trong NLP và nền tảng cho các mô hình trí tuệ nhân tạo hiện đại.

2.7.2. Kiến trúc Transformer

Mọi mô hình Transformer dùng để sinh văn bản đều bao gồm ba thành phần chính sau:

1. **Embedding:** Văn bản đầu vào được chia thành các đơn vị nhỏ hơn gọi là *token*, có thể là từ hoặc các từ con (subword). Các token này được chuyển thành các véc-tơ số gọi là *embedding*, giúp mô hình hiểu được ý nghĩa ngữ nghĩa của từ.
2. **Transformer Block:** là khối xây dựng cơ bản của mô hình, chịu trách nhiệm xử lý và biến đổi dữ liệu đầu vào. Mỗi khối bao gồm:
 - **Attention Mechanism:** Đây là thành phần cốt lõi của khối Transformer. Nó cho phép các token “giao tiếp” với nhau, từ đó mô hình nắm bắt ngữ cảnh và mối quan hệ giữa các từ trong câu.
 - **MLP (Multilayer Perceptron) Layer:** Đây là mạng nơ-ron truyền thẳng hoạt động độc lập trên từng token. Trong khi mục tiêu của lớp attention là trao đổi và truyền thông tin giữa các token, mục tiêu của lớp MLP là tinh chỉnh và làm giàu biểu diễn của từng token.
3. **Output Probabilities:** Các lớp tuyến tính và softmax ở cuối mô hình sẽ chuyển các embedding đã được xử lý thành các xác suất. Nhờ đó, mô hình có thể dự đoán token tiếp theo trong chuỗi.



Hình 2.5: Transformer Architecture

3. Mô tả bài toán

3.1. Bài toán sinh văn bản tự động

Bài toán sinh văn bản tự động (Text Generation) thuộc loại bài toán dự đoán chuỗi tuần tự, trong đó mô hình học cách tạo ra các token tiếp theo dựa trên bối cảnh của các token trước đó. Mục tiêu là xây dựng một mô hình có khả năng sinh ra văn bản tự nhiên, giữ được ngữ pháp, cú pháp, và mối quan hệ ngữ nghĩa trong câu.

Trong dự án GPT mini, bài toán được cụ thể hóa thành dự đoán token tiếp theo theo dạng character-level, nghĩa là mỗi token là một ký tự trong văn bản. Mỗi ký tự đầu vào được mã hóa thành vector embedding, kết hợp với positional embedding để giữ thông tin thứ tự, rồi đưa qua các block Transformer. Mô hình học phân phối xác suất của ký tự tiếp theo dựa trên bối cảnh trước đó.

Quá trình sinh văn bản thực hiện bằng cách cung cấp một đoạn văn bản đầu vào (prompt), từ đó mô hình dự đoán ký tự tiếp theo, thêm ký tự đó vào chuỗi và lặp lại quá trình cho đến khi đạt độ dài mong muốn. Bài toán này vừa yêu cầu khả năng nắm bắt ngữ cảnh cục bộ (token gần nhau) vừa phải học được quan hệ dài hạn giữa các ký tự trong chuỗi.

Điểm đặc trưng của bài toán sinh văn bản tự động là khả năng tạo nội dung mới dựa trên bối cảnh học được từ dữ liệu huấn luyện, không chỉ sao chép mà còn tạo ra các chuỗi ký tự hợp lý, giống với cách con người viết văn.

3.2. Dữ liệu huấn luyện

Dữ liệu huấn luyện sử dụng trong dự án GPT mini là văn bản từ cuốn The Wizard of Oz. Đây là một tập dữ liệu dạng văn bản thuần (plain text) tiếng Anh, bao gồm các ký tự chữ, số, dấu câu và khoảng trắng. Tập dữ liệu này được xử lý để xây dựng bộ từ vựng (vocabulary) bao gồm tất cả các ký tự xuất hiện trong văn bản.

Việc lựa chọn dữ liệu từ một tác phẩm văn học giúp mô hình học được cấu trúc ngôn ngữ tự nhiên, ngữ pháp, cách đặt câu, cũng như các mẫu ngôn ngữ đặc trưng. Dữ liệu được chia thành các phần để huấn luyện (train) và kiểm tra/đánh giá (validation), đồng thời áp dụng kỹ thuật memory mapping để lấy các đoạn văn bản ngẫu nhiên cho việc huấn luyện mà không cần tải toàn bộ tập dữ liệu vào bộ nhớ, giúp tiết kiệm tài nguyên.

3.3. Character-level prediction

Dự án GPT mini thực hiện bài toán sinh văn bản theo character-level, nghĩa là mỗi token đầu vào là một ký tự thay vì một từ. Mỗi ký tự được ánh xạ vào một chỉ số trong bộ từ vựng và sau đó biến đổi thành vector embedding. Mô hình học dự đoán ký tự tiếp theo dựa trên các ký tự trước đó, sử dụng cơ chế self-attention để nắm bắt mối quan hệ giữa tất cả các ký tự trong chuỗi.

Character-level prediction có ưu điểm là:

- Không phụ thuộc bộ từ vựng lớn, tránh vấn đề từ hiếm (rare words).
- Giúp mô hình học các mẫu cú pháp và ngữ pháp chi tiết ở mức ký tự.
- Linh hoạt với các dữ liệu nhỏ và đa dạng, có thể sinh ra các từ hoặc cấu trúc chưa xuất hiện trong dữ liệu huấn luyện.

Tuy nhiên, nhược điểm là quá trình huấn luyện có thể chậm hơn so với word-level, vì chuỗi ký tự dài hơn và cần học được các quan hệ dài hạn giữa các ký tự để sinh ra văn bản mạch lạc.

Trong GPT mini, việc áp dụng character-level prediction kết hợp với cơ chế Transformer và self-attention cho phép mô hình sinh văn bản tự nhiên dựa trên prompt, học được phân phối xác suất của từng ký tự một cách chính xác.

4. Thiết kế hệ thống và mô hình

4.1. Pipeline tổng quát

Pipeline của hệ thống GPT Mini được thiết kế theo trình tự logic từ dữ liệu đầu vào đến sinh văn bản:

1. **Đọc dữ liệu:** Văn bản thô (wizard_oz.txt) được đọc vào bộ nhớ, chuẩn bị cho việc token hóa.
2. **Token hóa và ánh xạ:** Xây dựng bảng vocab gồm tất cả ký tự xuất hiện trong văn bản. Mỗi ký tự được ánh xạ sang token số nguyên (index), phục vụ cho mô hình xử lý.
3. **Tạo batch:** Các đoạn văn bản được chia thành các batch ngẫu nhiên với độ dài block_size. Input x là chuỗi token, target y là cùng chuỗi dịch sang phải một token, dùng để huấn luyện dự đoán token tiếp theo.
4. **Embedding:** Input token được biến đổi thành vector liên tục qua token embedding, kết hợp với positional embedding để giữ thông tin thứ tự.
5. **Transformer block:** Các embedding đầu vào đi qua các block Transformer, mỗi block gồm Multi-Head Self-Attention và Feed Forward với LayerNorm, giúp học mối quan hệ giữa các token và biểu diễn ngữ cảnh.
6. **Dự đoán và loss:** Output của Transformer được đưa qua linear layer để dự đoán xác suất token tiếp theo. Loss được tính bằng Cross-Entropy so với target y .
7. **Huấn luyện:** Mô hình được cập nhật bằng AdamW optimizer. Loss trên batch validation được đánh giá định kỳ.
8. **Sinh văn bản:** Mô hình có thể sinh văn bản tự động từ prompt ban đầu bằng cách dự đoán tuần tự các token tiếp theo, cập nhật context liên tục.

4.2. Tiền xử lý dữ liệu

Tiền xử lý dữ liệu được tổ chức logic để tối ưu hóa cho huấn luyện:

1. **Đọc và ánh xạ bộ nhớ:** Dữ liệu được đọc bằng mmap để trích xuất đoạn văn bản bất kỳ, tiết kiệm bộ nhớ và xử lý file lớn.
2. **Chuẩn hóa dữ liệu:** Loại bỏ ký tự không hợp lệ, thống nhất định dạng, đảm bảo tính đồng nhất của văn bản.
3. **Xây dựng vocab:** Tập hợp tất cả ký tự xuất hiện trong văn bản, tạo mapping char $\leftrightarrow$ token bằng các hàm encode và decode.
4. **Tạo batch huấn luyện:** Chọn ngẫu nhiên các đoạn văn bản làm batch, input x và target y , mỗi batch có kích thước $\text{batch_size} \times \text{block_size}$.
5. **Chuyển về device:** Batch được chuyển về device phù hợp (CPU, GPU hoặc MPS), đảm bảo xử lý hiệu quả, giữ nguyên ngữ cảnh và thứ tự token để mô hình học phân phối xác suất chính xác.

4.3. Kiến trúc mô hình GPT Mini

Kiến trúc GPT Mini được thiết kế theo dạng Transformer nhỏ, tập trung vào việc học phân phối xác suất của từng ký tự và sinh văn bản tự động. Các thành phần logic và mối liên hệ giữa chúng được mô tả như sau:

1. **Token Embedding:** Chuyển các token rời rạc (ký tự) sang vector liên tục với kích thước n_embd . Đây là bước đầu để mô hình có thể học đặc trưng ngữ nghĩa của từng ký tự.
2. **Positional Embedding:** Thêm thông tin vị trí cho mỗi token trong chuỗi, giúp mô hình nhận biết thứ tự các ký tự và duy trì ngữ cảnh.
3. **Cộng Embedding:** Kết hợp token embedding và positional embedding để tạo đầu vào giàu thông tin cho Transformer block.
4. **Transformer Block:**
 - **Multi-Head Self-Attention:** Mỗi head học các mối quan hệ khác nhau giữa các token trong cùng một chuỗi. Các head được ghép lại và biến đổi qua linear layer để giữ cùng kích thước embedding ban đầu.

- **Feed Forward Layer:** Học các biểu diễn phi tuyến, tăng khả năng mô hình hóa các mối quan hệ phức tạp giữa token.
 - **LayerNorm và Residual Connection:** Giúp ổn định quá trình huấn luyện, tránh mất mát thông tin khi đi qua các block.
5. **Stack các Block:** Trong GPT Mini, chỉ sử dụng 1 block (`n_layer = 1`) với 8 head attention (`n_head = 8`) và embedding `n_embd = 128`.
 6. **LayerNorm cuối cùng và Linear Head:** Output cuối cùng được chuẩn hóa và đưa qua linear layer dự đoán xác suất token tiếp theo.
 7. **Sinh văn bản:** Mô hình sử dụng đầu ra xác suất để sinh token mới tuần tự, cập nhật context liên tục, cho phép tạo ra văn bản tự nhiên theo prompt đầu vào.

4.4. Siêu tham số

Các siêu tham số trong GPT Mini được lựa chọn để cân bằng giữa khả năng học và chi phí tính toán:

- `batch_size = 8`: Số lượng chuỗi token được huấn luyện đồng thời.
- `block_size = 32`: Chiều dài tối đa của chuỗi token mà mô hình nhìn thấy mỗi lần.
- `n_embd = 128`: Kích thước embedding của token và positional embedding.
- `n_head = 8`: Số lượng head trong Multi-Head Attention, mỗi head học các mối quan hệ khác nhau giữa các token.
- `n_layer = 1`: Số lượng Transformer block, trong GPT Mini chỉ dùng 1 block để giảm độ phức tạp.
- `dropout = 0.2`: Tỷ lệ dropout để giảm overfitting.
- `learning_rate = 1e-4`: Tốc độ học cho bộ tối ưu AdamW.
- `max_iters = 1000`: Số vòng lặp huấn luyện tối đa.

- `eval_iters = 100`: Số batch dùng để ước lượng loss trung bình khi đánh giá.

Các siêu tham số này quyết định cách mô hình học, độ ổn định và chất lượng sinh văn bản, đồng thời phản ánh giới hạn tài nguyên và mục tiêu mô hình nhỏ gọn của GPT Mini.

5. Huấn luyện, thực nghiệm và đánh giá

5.1. Quy trình huấn luyện

Quy trình huấn luyện mô hình GPT Mini được triển khai theo các bước logic sau:

Thiết lập môi trường và siêu tham số

- Sử dụng PyTorch, chạy trên GPU nếu có (`device = 'cuda'`).
- Các siêu tham số chính: `batch_size = 8, block_size = 32, max_iters = 1000, learning_rate = 1e-4, eval_iters = 100, n_embd = 128, n_head = 8, n_layer = 1, dropout = 0.2`.

Chuẩn bị dữ liệu

- Đọc file văn bản gốc "wizard_of_oz.txt" và file vocab "vocab.txt".
- Xây dựng từ điển ánh xạ ký tự $\leftrightarrow$ số (`string_to_int` và `int_to_string`).
- Mỗi batch được tạo bằng hàm `get_batch(split)` sử dụng `mmap` để lấy ngẫu nhiên các đoạn văn bản dài `block_size * batch_size`, sau đó tách thành input x và target y (dịch chuyển một token).

Khởi tạo mô hình GPT Mini

- Embedding cho token và vị trí (`nn.Embedding`).
- Transformer block: Multi-Head Self-Attention + FeedForward + LayerNorm.
- Số layer: 1, số head: 8, embedding size: 128.
- Head cuối cùng (`lm_head`) xuất logits có kích thước bằng `vocab_size`.

Bộ tối ưu và hàm mất mát

- Sử dụng Cross-Entropy Loss.
- Bộ tối ưu AdamW với learning rate `1e-4`.

Training loop

Lặp max_iters lần:

1. Lấy batch dữ liệu train.
2. Forward pass qua mô hình $\rightarrow$ logits $\rightarrow$ loss.
3. Zero gradients $\rightarrow$ backward $\rightarrow$ cập nhật trọng số với AdamW.
4. Định kỳ (eval_iters) đánh giá loss trung bình trên train và validation để theo dõi quá trình huấn luyện.

Lưu mô hình và sinh văn bản thử nghiệm

- Sau khi huấn luyện xong, mô hình được lưu bằng pickle.
- Sinh văn bản từ prompt, ví dụ 'Hello! Can you see me?' với 100 token mới, thể hiện khả năng mô hình học phân phối ký tự từ dữ liệu.

Kết quả huấn luyện sơ bộ

- Loss giảm dần: train loss từ $\sim 8.63 \rightarrow 2.65$, val loss từ $\sim 8.63 \rightarrow 2.63$, mô hình hội tụ ổn định.
- Văn bản sinh từ mô hình giữ cấu trúc cơ bản, xuất hiện một số ký tự lạ do block_size và vocab hạn chế.

5.2. Lập trình training loop và batch generation

Quy trình lập trình training loop và sinh batch dữ liệu được triển khai trực tiếp từ code như sau:

Batch generation

- Hàm get_random_chunk(split) dùng mmap đọc ngẫu nhiên một đoạn văn bản từ file wizard_of_oz.txt có độ dài block_size * batch_size.

- Đoạn văn bản này được encode thành số nguyên dựa trên từ điển `string_to_int`.
- Hàm `get_batch(split)`:
 - Chọn ngẫu nhiên `batch_size` vị trí trong đoạn văn bản.
 - Tạo input x gồm các chuỗi dài `block_size`.
 - Tạo target y là dịch chuyển x sang phải một token (next-character prediction).
 - Cả x và y được chuyển sang device (cuda hoặc cpu) để huấn luyện.

Training loop

- Lặp `max_iters` lần:
 1. Đánh giá loss định kỳ: gọi hàm `estimate_loss()` với `eval_iters = 100`, tính trung bình train loss và val loss.
 2. Lấy batch dữ liệu: `xb, yb = get_batch('train')`.
 3. Forward pass: `logits, loss = model.forward(xb, yb)`.
 4. Backward pass: `optimizer.zero_grad(set_to_none=True), loss.backward(), optimizer.step()`.
 5. In ra tiến trình: `step`, train loss và val loss để theo dõi sự hội tụ.

Logic quan trọng

- `estimate_loss()` chạy `torch.no_grad()` để tính loss mà không cập nhật gradient, giúp đánh giá định kỳ mà không làm thay đổi trọng số.
- Batch được tạo ngẫu nhiên ở mỗi iteration để tăng tính đa dạng dữ liệu, tránh overfitting.

5.3. Đánh giá loss định kỳ

Đánh giá loss định kỳ giúp theo dõi quá trình huấn luyện và quyết định khi nào mô hình hội tụ:

- Sử dụng hàm `estimate_loss()`:

- Chạy mô hình ở chế độ eval() để tắt dropout.
- Lặp qua eval_iters batch, tính trung bình cross-entropy loss cho train và validation.
- Sau khi tính xong, chuyển lại mô hình sang chế độ train().
- Output mẫu (iteration đầu và cuối):

```
step: 0, train loss: 8.633, val loss: 8.633
step: 1, train loss: 8.618, val loss: 8.616
...
step: 999, train loss: 2.652, val loss: 2.626
```

- Loss giảm dần chứng tỏ mô hình học tốt phân phối ký tự.
- Chênh lệch nhỏ giữa train loss và val loss cho thấy mô hình không bị overfitting nghiêm trọng.

5.4. Kết quả huấn luyện và sinh văn bản

Kết quả huấn luyện

- Train loss: từ $\sim 8.63 \rightarrow 2.65$.
- Val loss: từ $\sim 8.63 \rightarrow 2.63$.
- Mô hình hội tụ ổn định trong 1000 iteration với batch_size nhỏ (8) và block_size 32.

Ví dụ sinh văn bản

- Prompt: 'Hello! Can you see me?'
- Sinh 100 token mới, output:

```
Hello! Can you see me?Pwhetomudhef owe at tBy seeasg t ithreulisga
u hedl me d heo
ulei
o sereehupp ocepuirof;er
```

- Văn bản giữ cấu trúc cơ bản, một số ký tự lạ xuất hiện do tập dữ liệu nhỏ và vocab giới hạn.

Nhận xét và đánh giá chất lượng mô hình

- Mô hình GPT Mini 1 layer, 8 head, embedding 128 học được phân phối ký tự cơ bản.
- Loss giảm đều, sinh văn bản có cấu trúc từ ngữ hợp lý nhưng vẫn xuất hiện noise.
- Nếu tăng số layer, embedding, dataset lớn hơn, chất lượng sinh văn bản sẽ cải thiện đáng kể.

5.5. Kết luận và hướng phát triển

Kết luận

Từ báo cáo này, có thể rút ra một số kết luận quan trọng và có ý nghĩa đối với quá trình xây dựng mô hình ngôn ngữ lớn (LLM). Trước hết, việc nắm vững và áp dụng đúng các cơ sở lý thuyết như Tensor, cơ chế tính toán tự động (autograd), các phương pháp tối ưu hoá như Gradient Descent, cùng các thành phần cấu trúc của mô hình Transformer, đóng vai trò nền tảng không thể thiếu. Những kiến thức này không chỉ giúp người nghiên cứu triển khai mô hình đúng hướng, mà còn tạo điều kiện để phân tích, đánh giá và cải thiện mô hình trong suốt vòng đời phát triển.

Bên cạnh đó, kiến trúc Transformer và cơ chế Attention được khẳng định là trung tâm của mọi mô hình LLM hiện đại. Nhờ khả năng học mối quan hệ dài hạn giữa các token và xử lý song song hiệu quả, Transformer đã chứng minh vượt trội hơn hẳn so với các mô hình truyền thống như RNN hay LSTM, từ đó nâng cao hiệu năng và chất lượng sinh văn bản. Việc hiểu sâu cách thức hoạt động của Self-Attention, Multi-Head Attention và Position Embedding là yếu tố quyết định để tối ưu hoá mô hình và tránh các sai sót trong thiết kế.

Ngoài ra, pipeline huấn luyện hoàn chỉnh – bao gồm bước chuẩn bị dữ liệu, mã hoá dữ liệu, lựa chọn siêu tham số, xây dựng kiến trúc mô hình, triển khai vòng lặp huấn luyện, quản lý batch và đánh giá loss định kỳ – có ảnh hưởng trực tiếp đến chất lượng kết quả đầu ra. Một pipeline hợp lý sẽ giúp mô hình học ổn định, tránh overfitting hoặc underfitting, đồng thời tối ưu hoá tốc độ huấn luyện và hiệu quả sử dụng tài nguyên.

Không thể không nhắc đến PyTorch, nền tảng đóng vai trò then chốt trong suốt quá trình nghiên cứu và thực nghiệm. Nhờ tính linh hoạt, khả năng quản lý gradient mạnh mẽ, hỗ trợ tính toán GPU và việc cho phép tùy chỉnh mô hình dễ dàng, PyTorch trở thành công cụ tối ưu cho việc xây dựng và mở rộng các mô hình ngôn ngữ lớn. Hệ sinh thái rộng, tài liệu phong phú và cộng đồng phát triển mạnh cũng góp phần hỗ trợ quá trình nghiên cứu diễn ra thuận lợi hơn.

Tổng kết lại, đề tài đã chứng minh khả năng thiết kế, triển khai và huấn luyện một mô hình LLM thu nhỏ (GPT Mini) dựa trên Transformer, với hiệu quả sinh văn bản tương đối tốt so với quy mô mô hình. Điều này không chỉ khẳng định tính khả thi của việc xây dựng LLM trên tài nguyên hạn chế, mà còn cung cấp nền tảng vững chắc cho những hướng phát triển tiếp theo. Các kết quả thu được giúp mở ra triển vọng phát triển các mô hình lớn hơn, cải thiện chất lượng sinh văn bản, tối ưu hoá hiệu năng, và mở rộng ứng dụng của LLM trong nhiều lĩnh vực như chatbot, phân tích văn bản, dịch máy, tóm tắt nội dung và nhiều tác vụ NLP hiện đại khác.

Hướng phát triển

Dựa trên kết quả đạt được, một số hướng phát triển tiềm năng cho nghiên cứu và ứng dụng LLM bao gồm:

1. **Mở rộng quy mô mô hình:** Áp dụng kiến trúc Transformer lớn hơn, tăng số lớp và số chiều embedding để cải thiện khả năng học ngữ nghĩa phức tạp và nâng cao chất lượng sinh văn bản.
2. **Cải tiến dữ liệu huấn luyện:** Sử dụng các tập dữ liệu đa dạng và lớn hơn, bao gồm dữ liệu song ngữ hoặc dữ liệu chuyên ngành, để mô hình sinh văn bản chính xác hơn trong nhiều ngữ cảnh khác nhau.
3. **Tối ưu hóa huấn luyện:** Áp dụng các kỹ thuật tối ưu nâng cao như AdamW, learning rate scheduler, gradient clipping hoặc mixed precision training để tăng tốc huấn luyện và giảm thiểu overfitting.
4. **Triển khai và ứng dụng thực tiễn:** Phát triển API hoặc chatbot dựa trên LLM, áp dụng cho trợ lý ảo, dịch tự động, sinh văn bản sáng tạo, và các hệ thống hỗ trợ người dùng thông minh.

5. **Nghiên cứu cơ chế Attention nâng cao:** Khai thác các kỹ thuật Attention mới, như Sparse Attention hay FlashAttention, nhằm tăng hiệu quả tính toán cho mô hình lớn, đặc biệt khi triển khai LLM trên tập dữ liệu cực lớn.
6. **Khả năng mở rộng đa thiết bị:** Nghiên cứu phân tán mô hình (model parallelism, data parallelism) để triển khai LLM trên nhiều GPU hoặc cluster, phục vụ huấn luyện mô hình quy mô lớn.

Tổng kết lại, đề tài không chỉ chứng minh được khả năng xây dựng, huấn luyện và đánh giá một mô hình LLM nhỏ, mà còn tạo nền tảng để tiếp tục mở rộng, tối ưu và ứng dụng các mô hình ngôn ngữ lớn trong các bài toán thực tế, góp phần phát triển các hệ thống AI thế hệ mới.

Tài liệu tham khảo

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*.
- [2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI.
- [3] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
- [4] Karpathy, A. (2021). *nanoGPT: Minimal PyTorch Implementation of GPT*. GitHub.
- [5] PyTorch Documentation (2025).